

1. Cargar datos

```

pip install plotly
!pip install kneed
!pip install dash

# Cambiar el backend de pandas para usar plotly
import pandas as pd

# Asegurarse de que pandas esté configurado para usar plotly
pd.options.plotting.backend = "plotly"

# Cargamos otras librerías
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
from kneed import KneeLocator
from sklearn.svm import SVR
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import os
from dash import Dash, dcc, html, Input, Output
from sklearn.metrics import mean_squared_error
import plotly.express as px
import plotly.graph_objects as go

# Cargar los datos usamos OS para facilidad de todos

file_name = 'Datos Contugas 2.xlsx'
file_path = os.path.abspath(file_name)
excel_data = pd.ExcelFile(file_path)
df_combined = pd.DataFrame()

df_combined.head()

```

2. Pre-procesamiento de datos

```

# Iterar sobre cada hoja y agregar el número de cliente ya que están separados
for i, sheet_name in enumerate(excel_data.sheet_names, start=1):
    df_temp = excel_data.parse(sheet_name)
    df_temp['Numero_Cliente'] = f'CLIENTE{i}'
    df_combined = pd.concat([df_combined, df_temp], ignore_index=True)

# Fecha a datetime
df_combined['Fecha'] = pd.to_datetime(df_combined['Fecha'])
df_combined

# Agregamos una columna de mes para agregar variable de estacionalidad y otras variables como semana, día, o si es fin de semana o no.
df_combined['Mes'] = df_combined['Fecha'].dt.month
df_combined['dia_semana'] = df_combined['Fecha'].dt.dayofweek
df_combined['semana_año'] = df_combined['Fecha'].dt.isocalendar().week
df_combined['es_fin_de_semana'] = df_combined['dia_semana'].apply(lambda x: 1 if x >= 5 else 0)

import pandas as pd
from scipy.stats import pearsonr

# Supongamos que ya cargaste tu dataframe
# df_combined = pd.read_csv('archivo.csv') # o como lo hayas cargado

# Lista para guardar resultados
resultados = []

# Iterar por cada cliente
for cliente, grupo in df_combined.groupby("Numero_Cliente"):
    # Eliminar filas con NaN
    grupo = grupo.dropna(subset=["Presion", "Temperatura", "Volumen"])

```

```

if len(grupo) > 1:
    # Correlación Presión vs Volumen
    corr_pv, pval_pv = pearsonr(grupo["Presion"], grupo["Volumen"])

    # Correlación Temperatura vs Volumen
    corr_tv, pval_tv = pearsonr(grupo["Temperatura"], grupo["Volumen"])

    resultados.append({
        "Numero_Cliente": cliente,
        "Corr_Presion_Volumen": corr_pv,
        "p_Presion_Volumen": pval_pv,
        "Corr_Temperatura_Volumen": corr_tv,
        "p_Temperatura_Volumen": pval_tv
    })

# Convertir a DataFrame
df_correlaciones = pd.DataFrame(resultados)

df_correlaciones

# Completitud
completitud_por_cliente = df_combined.groupby('Numero_Cliente').apply(lambda x: x.notnull().mean() * 100)

# Crear gráfico de barras con plotly
fig = px.bar(completitud_por_cliente,
              title="Completitud por Cliente",
              labels={'value': 'Completitud (%)'})

# Ajustar tamaño de la figura
fig.update_layout(
    width=800, # Ancho de la figura
    height=600 # Alto de la figura
)

# Mostrar el gráfico
fig.show()

#Consistencia y claridad
# Obtener estadísticas descriptivas para todas las columnas numéricas
estadisticas_descriptivas = df_combined.describe()

# Convertir el DataFrame a formato tabular (texto plano)
tabla_texto = estadisticas_descriptivas.to_string()

# Imprimir la tabla para copiarla fácilmente
print(tabla_texto)

##Formato
# Obtener los tipos de datos (formato) de cada columna
tipos_de_datos = df_combined.dtypes

# Convertir los tipos de datos a una tabla legible
tabla_formatos = tipos_de_datos.to_frame().reset_index()

# Renombrar las columnas para mayor claridad
tabla_formatos.columns = ['Variable', 'Formato']

# Convertir la tabla a un formato de texto tabular (para copiar y pegar)
tabla_formatos_texto = tabla_formatos.to_string(index=False)

# Imprimir la tabla para copiarla
print(tabla_formatos_texto)

import plotly.express as px

# Crear un scatter plot de Temperatura contra Fecha
fig = px.scatter(df_combined,
                 x='Fecha',
                 y='Temperatura',
                 title='Temperatura a lo largo del tiempo',
                 labels={'Fecha': 'Fecha', 'Temperatura': 'Temperatura (°C)'},
                 template='plotly_dark')

```

```

# Mostrar el gráfico
fig.show()

import plotly.express as px

# Scatter plot para Volumen vs. Fecha (Tiempo)
fig_volumen = px.scatter(df_combined,
                        x='Fecha',
                        y='Volumen',
                        title='Volumen a lo largo del tiempo',
                        labels={'Fecha': 'Fecha', 'Volumen': 'Volumen (m³)'},
                        template='plotly_dark')

# Scatter plot para Presión vs. Fecha (Tiempo)
fig_presion = px.scatter(df_combined,
                        x='Fecha',
                        y='Presion',
                        title='Presión a lo largo del tiempo',
                        labels={'Fecha': 'Fecha', 'Presion': 'Presión (Pa)'},
                        template='plotly_dark')

# Mostrar ambos gráficos
fig_volumen.show()
fig_presion.show()

from plotly.subplots import make_subplots
import plotly.graph_objects as go
# Crear una figura con tres subplots (uno para cada variable)
fig = make_subplots(rows=1, cols=3, subplot_titles=["Presion", "Temperatura", "Volumen"])

# Creamos un DF para saber que estan clientes estan disponibles
clientes = df_combined['Numero_Cliente'].unique()
cliente_inicial = clientes[0]

# Añadir los gráficos de caja para el cliente inicial
for i, variable in enumerate(['Presion', 'Temperatura', 'Volumen']):
    fig.add_trace(
        go.Box(y=df_combined[df_combined['Numero_Cliente'] == cliente_inicial][variable],
              name=variable,
              boxpoints='all'),
        row=1, col=i+1
    )

# botones para el menú desplegable
buttons = []
for cliente in clientes:
    buttons.append(dict(
        label=cliente,
        method="update",
        args=[
            {"y": [
                df_combined[df_combined['Numero_Cliente'] == cliente]['Presion'],
                df_combined[df_combined['Numero_Cliente'] == cliente]['Temperatura'],
                df_combined[df_combined['Numero_Cliente'] == cliente]['Volumen']
            ]},
            {"title": f"Distribución de Presion, Temperatura y Volumen para {cliente}"}
        ]
    ))

# menú desplegable al layout
fig.update_layout(
    updatemenus=[dict(
        buttons=buttons,
        direction="down",
        showactive=True,
        x=0.5,
        xanchor="center",
        y=1.2,
        yanchor="top"
    )]
)

#
fig.update_layout(
    title=f"Distribución de Presion, Temperatura y Volumen para {cliente_inicial}",
    yaxis_title="Valor",

```

```

)

#
fig.show()

import plotly.graph_objects as go
import pandas as pd

df_combined['Fecha'] = pd.to_datetime(df_combined['Fecha'])

for cliente in df_combined['Numero_Cliente'].unique():
    df_cliente = df_combined[df_combined['Numero_Cliente'] == cliente]

    fig = go.Figure()
    fig.add_trace(go.Scatter(x=df_cliente['Fecha'], y=df_cliente['Presion'], mode='lines', name='Presion'))
    fig.add_trace(go.Scatter(x=df_cliente['Fecha'], y=df_cliente['Temperatura'], mode='lines', name='Temperatura'))
    fig.add_trace(go.Scatter(x=df_cliente['Fecha'], y=df_cliente['Volumen'], mode='lines', name='Volumen'))

    fig.update_layout(
        title=f'Presión, Temperatura y Volumen para {cliente}',
        xaxis_title='Fecha',
        yaxis_title='Valor',
        width=900,
        height=500
    )

    fig.show()

```

3. Entrenamos los modelos

3.4 Modelo 3: DBSCAN con Regresión SVR Mejorado

```

#Función de cálculo de métricas de desempeño sin escalar
def metricas_desempeno_dbscan_1(df_resultado, features):
    resultados_metricas = []

    for cliente_id, grupo in df_resultado.groupby('Numero_Cliente'):
        grupo_clusters = grupo[grupo['cluster_dbscan'] != -1] # Excluir outliers

        if len(grupo_clusters['cluster_dbscan'].unique()) < 2:
            # No se puede calcular métricas con menos de 2 clusters
            silhouette = np.nan
            davies_bouldin = np.nan
            calinski_harabasz = np.nan
        else:
            X = grupo_clusters[features]
            labels = grupo_clusters['cluster_dbscan']

            silhouette = silhouette_score(X, labels)
            davies_bouldin = davies_bouldin_score(X, labels)
            calinski_harabasz = calinski_harabasz_score(X, labels)

        resultados_metricas.append({
            'Numero_Cliente': cliente_id,
            'Silhouette Score': silhouette,
            'Davies-Bouldin Score': davies_bouldin,
            'Calinski-Harabasz Score': calinski_harabasz
        })

    return pd.DataFrame(resultados_metricas)

#Función de cálculo de métricas de desempeño escalando
def metricas_desempeno_dbscan_2(df_resultado, features):
    resultados_metricas = []
    scaler = StandardScaler()

    for cliente_id, grupo in df_resultado.groupby('Numero_Cliente'):
        grupo_clusters = grupo[grupo['cluster_dbscan'] != -1] # Excluir outliers

        if len(grupo_clusters['cluster_dbscan'].unique()) < 2:
            silhouette = np.nan
            davies_bouldin = np.nan
            calinski_harabasz = np.nan
        else:

```

```

X = grupo_clusters[features]
X_scaled = scaler.fit_transform(X) # Escalado
labels = grupo_clusters['cluster_dbscan']

silhouette = silhouette_score(X_scaled, labels)
davies_bouldin = davies_bouldin_score(X_scaled, labels)
calinski_harabasz = calinski_harabasz_score(X_scaled, labels)

resultados_metricas.append({
    'Numero_Cliente': cliente_id,
    'Silhouette Score': silhouette,
    'Davies-Bouldin Score': davies_bouldin,
    'Calinski-Harabasz Score': calinski_harabasz
})

return pd.DataFrame(resultados_metricas)

#miramos el numero de clusters
def numero_clusters(df, columna_cluster='cluster_dbscan'):
    clusters_por_cliente = df.groupby('Numero_Cliente')[columna_cluster] \
        .nunique().reset_index(name='Numero_Clusters')
    return print(clusters_por_cliente)

def modelo_hibrido_svr_dbscan_2(df, usar_temperatura=True, usar_presion=True, median_factor=1.5, max_iter=5, min_samples=8):
    df = df.copy()
    df['Fecha'] = pd.to_datetime(df['Fecha'])
    df = df.set_index('Fecha')
    df['Mes'] = df.index.month
    df['Dia_Semana'] = df.index.dayofweek

    resultados = []

    for cliente_id, grupo in df.groupby("Numero_Cliente"):
        grupo = grupo.sort_index()

        # Imputación inicial
        for var in ["Temperatura", "Presion"]:
            if var in grupo.columns:
                max_val = grupo[var].max(skipna=True)
                grupo[var] = grupo[var].fillna(100 * max_val)

        grupo['Volumen'] = grupo['Volumen'].fillna(100 * grupo['Volumen'].max(skipna=True))
        grupo['x_seq'] = np.arange(len(grupo))

        # Construcción dinámica de variables predictoras
        features = ['x_seq', 'Mes', 'Dia_Semana']
        if usar_temperatura and 'Temperatura' in grupo.columns:
            features.insert(0, 'Temperatura') # opcional: cambiar orden
        if usar_presion and 'Presion' in grupo.columns:
            features.insert(0, 'Presion')

        X_base = grupo[features]
        Y = grupo['Volumen'].copy()

        # --- Separar espacio de SVR ---
        # Usamos todas las features menos x_seq
        SVR_features = [col for col in features if col != 'x_seq']
        X_base_SVR=grupo[SVR_features]
        # --- Separar espacio de clustering ---
        # Usamos solo 'Mes', 'Dia_Semana', excluyendo Temperatura y presion para clustering
        clustering_features = ['x_seq', 'Mes', 'Dia_Semana']
        X_base_dbscan = grupo[clustering_features]

        outlier_filamentos_idx = set()
        final_labels = pd.Series(index=grupo.index, data=np.nan)
        umbral_residual = 0.05 * Y.max()

        for iteracion in range(max_iter):
            if Y.isna().any():
                max_y = Y.max(skipna=True)
                Y = Y.fillna(100 * max_y if not pd.isna(max_y) else 1e6)

            # Escalar para clustering
            scaler_X = StandardScaler()
            X_scaled = scaler_X.fit_transform(X_base_dbscan)

```

```

scaler_Y = StandardScaler()
Y_scaled = scaler_Y.fit_transform(Y.values.reshape(-1, 1))

XY_scaled = np.hstack([X_scaled, Y_scaled])

k = min(20, len(XY_scaled) - 1)
neighbors = NearestNeighbors(n_neighbors=k)
distances, _ = neighbors.fit(XY_scaled).kneighbors(XY_scaled)
distances_k = np.sort(distances[:, k - 1])
eps = np.percentile(distances_k, 95)

db = DBSCAN(eps=eps, min_samples=min_samples)
labels = db.fit_predict(XY_scaled)

inliers_mask = labels != -1
if inliers_mask.sum() < 5:
    break

X_train = X_base_SVR[inliers_mask]
Y_train = Y[inliers_mask]
svr = SVR()
svr.fit(X_train, Y_train)
Y_pred = svr.predict(X_train)
residuals = np.abs(Y_train - Y_pred)

df_temp = pd.DataFrame({
    'label': labels[inliers_mask],
    'residual': residuals
})
cluster_residual_mean = df_temp.groupby('label')['residual'].mean()

if cluster_residual_mean.empty:
    break

top_cluster = cluster_residual_mean.idxmax()
top_residual = cluster_residual_mean.max()

if top_residual < umbral_residual:
    break

cluster_mask = (labels == top_cluster)
cluster_idx = grupo.index[cluster_mask]
outlier_filamentos_idx.update(cluster_idx)
Y.loc[cluster_idx] = np.nan

X_final_train = X_base_SVR.drop(index=outlier_filamentos_idx)
Y_final_train = Y.drop(index=outlier_filamentos_idx)
modelo_final = SVR()
modelo_final.fit(X_final_train, Y_final_train)

X_to_predict = X_base_SVR.copy()
Y_filled = Y.copy()
if Y_filled.isna().any():
    max_y = Y_filled.max(skipna=True)
    Y_filled = Y_filled.fillna(100 * max_y if not pd.isna(max_y) else 1e6)

Y_pred_final = modelo_final.predict(X_to_predict)
mse = mean_squared_error(Y_filled, Y_pred_final)

grupo_resultado = grupo.copy()
grupo_resultado['Volumen_Predicho'] = Y_pred_final
grupo_resultado['MSE'] = mse
grupo_resultado['outlier'] = False
grupo_resultado.loc[list(outlier_filamentos_idx), 'outlier'] = True

# Reescalar y reetiquetar con DBSCAN final
val_mask = X_to_predict.notnull().all(axis=1) & Y_filled.notnull()

# solo usar ['Mes', 'Dia_Semana'] para DBSCAN
X_valid_dbscan = grupo.loc[val_mask, clustering_features]
Y_valid = Y_filled[val_mask]

XY_scaled_pred = np.hstack([
    scaler_X.transform(X_valid_dbscan),
    scaler_Y.transform(Y_valid.values.reshape(-1, 1))
])

```

```

labels_pred = db.fit_predict(XY_scaled_pred)

final_labels_partial = pd.Series(index=X_valid_dbscan.index, data=labels_pred)
final_labels.update(final_labels_partial)

grupo_resultado['cluster_dbscan'] = final_labels

resultados.append(grupo_resultado)
# Notificación al terminar cada cliente
print(f"CLIENTE {cliente_id} -> DONE")
return pd.concat(resultados)

def dashboard_outliers_SVR(df_resultado):
    import pandas as pd
    from dash import Dash, html, dcc, Input, Output
    import plotly.graph_objects as go

    df_resultado = df_resultado.copy()
    df_resultado.index = pd.to_datetime(df_resultado.index)
    df_resultado['Año'] = df_resultado.index.year
    df_resultado['Mes'] = df_resultado.index.month

    app = Dash(__name__)

    app.layout = html.Div([
        html.H2("Volumen de Gas vs Tiempo por Cliente (Outliers y Clusters Destacados)"),

        html.Div([
            html.Label("Cliente:"),
            dcc.Dropdown(
                id='cliente-dropdown',
                options=[{'label': c, 'value': c} for c in sorted(df_resultado['Numero_Cliente'].unique())],
                value=df_resultado['Numero_Cliente'].iloc[0],
                clearable=False
            ),
        ], style={'width': '30%', 'display': 'inline-block'}),

        html.Div([
            html.Label("Año:"),
            dcc.Dropdown(id='año-dropdown', clearable=True),
        ], style={'width': '20%', 'display': 'inline-block'}),

        html.Div([
            html.Label("Mes:"),
            dcc.Dropdown(id='mes-dropdown', clearable=True),
        ], style={'width': '20%', 'display': 'inline-block'}),

        dcc.Graph(id='grafico-volumen-tiempo')
    ])

    @app.callback(
        Output('año-dropdown', 'options'),
        Input('cliente-dropdown', 'value')
    )
    def actualizar_años(cliente):
        años = df_resultado[df_resultado['Numero_Cliente'] == cliente]['Año'].unique()
        return [{'label': str(a), 'value': a} for a in sorted(años)]

    @app.callback(
        Output('mes-dropdown', 'options'),
        Input('cliente-dropdown', 'value'),
        Input('año-dropdown', 'value')
    )
    def actualizar_meses(cliente, año):
        df_filtrado = df_resultado[df_resultado['Numero_Cliente'] == cliente]
        if año:
            df_filtrado = df_filtrado[df_filtrado['Año'] == año]
        meses = df_filtrado['Mes'].unique()
        return [{'label': str(m), 'value': m} for m in sorted(meses)]

    @app.callback(
        Output('grafico-volumen-tiempo', 'figure'),
        Input('cliente-dropdown', 'value'),
        Input('año-dropdown', 'value'),
        Input('mes-dropdown', 'value')
    )

```

```

def actualizar_grafico(cliente, año, mes):
    df_cliente = df_resultado[df_resultado['Numero_Cliente'] == cliente]
    if año:
        df_cliente = df_cliente[df_cliente['Año'] == año]
    if mes:
        df_cliente = df_cliente[df_cliente['Mes'] == mes]

    fig = go.Figure()

    # 1. Clusters (puntos normales)
    normales = df_cliente[(~df_cliente['outlier']) & (df_cliente['cluster_dbscan'] != -1)]
    fig.add_trace(go.Scatter(
        x=normales.index,
        y=normales['Volumen'],
        mode='markers',
        marker=dict(
            color=normales['cluster_dbscan'],
            colorscale='Turbo',
            size=6,
            opacity=1,
            colorbar=dict(
                title='Cluster',
                thickness=12,
                len=0.5,
                x=1.02,
                y=0.5
            )
        ),
        name='Clusters',
        showlegend=False
    ))

    # 2. Outliers por DBSCAN (ruido)
    outliers_dbscan = df_cliente[df_cliente['cluster_dbscan'] == -1]
    fig.add_trace(go.Scatter(
        x=outliers_dbscan.index,
        y=outliers_dbscan['Volumen'],
        mode='markers',
        name='Outliers DBSCAN',
        marker=dict(color='orange', size=10, symbol='triangle-up')
    ))

    # 3. Outliers por SVR
    outliers_svr = df_cliente[df_cliente['outlier']]
    fig.add_trace(go.Scatter(
        x=outliers_svr.index,
        y=outliers_svr['Volumen'],
        mode='markers',
        name='Outliers SVR',
        marker=dict(color='red', size=10, symbol='x')
    ))

    # 4. Línea de predicción SVR
    fig.add_trace(go.Scatter(
        x=df_cliente.index,
        y=df_cliente['Volumen_Predicho'],
        mode='lines',
        name='SVR Prediction',
        line=dict(color='black', width=2),
        opacity=0.4
    ))

    fig.update_layout(
        title=f'Clusters de Volumen vs Fecha para {cliente}',
        xaxis_title='Fecha',
        yaxis_title='Volumen',
        legend_title='Tipo de Punto',
        template='plotly_white',
        height=600,
        margin=dict(r=80)
    )

    return fig

return app

```



```

df_resultado_modelo_4=modelo_hibrido_svr_dbscan_2(df_combined, usar_temperatura=True, usar_presion=False)
df_resultado_modelo_4

# Selecciona un cliente para visualizar
cliente_id = df_resultado_modelo_4['Numero_Cliente'].iloc[0] # o reemplaza por un ID específico
df_cliente = df_resultado_modelo_4[df_resultado_modelo_4['Numero_Cliente'] == cliente_id]

# Ordenar por fecha
df_cliente = df_cliente.sort_index()

# Graficar
plt.figure(figsize=(14, 6))
plt.plot(df_cliente.index, df_cliente['Volumen'], label='Original TS', color='blue')
plt.plot(df_cliente.index, df_cliente['Volumen_Predicho'], label='SVR prediction', color='red')

# Marcar outliers
outliers = df_cliente[df_cliente['outlier'] == True]
plt.scatter(outliers.index, outliers['Volumen'], color='orange', label='Modelo outliers', zorder=5)

# Opcional: marcar "real outliers" si los tienes (como en la gráfica)
# plt.scatter(df_cliente.index[condición], df_cliente['Volumen'][condición], color='green', label='Real outliers')

plt.title(f'Predicción y detección de outliers - Cliente {cliente_id}')
plt.xlabel('Fecha')
plt.ylabel('Volumen')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

app_modelo4 = dashboard_outliers_SVR(df_resultado_modelo_4)
app_modelo4.run(debug=True)

#miramos el numero de clusters
numero_clusters(df_resultado_modelo_4)

##Calculamos las métricas de desempeño de este modelo (escalado)
metricas_modelo_4_2=metricas_desempeno_dbscan_2(df_resultado_modelo_4, features=['x_seq', 'Mes', 'dia_semana', 'Volumen'])
print(metricas_modelo_4_2)

##Métricas promedio del modelo:
Metricas_promedio_modelo_4_2 = metricas_modelo_4_2[['Silhouette Score', 'Davies-Bouldin Score', 'Calinski-Harabasz Score']].mean()

print("Promedio de métricas de desempeño:")
print(Metricas_promedio_modelo_4_2)

# Leer el archivo Excel
#df = pd.read_excel('resultado_modelo_4.xlsx')
#df_resultado_modelo_4=df

#Clasificación de los clusters en riesgo
df_resultado_modelo_4['Residual']=df_resultado_modelo_4['Volumen_Predicho']-df_resultado_modelo_4['Volumen']
def riesgo_cluster(df):
    # Filtramos los filamentos de outliers
    df_outliers = df[df['outlier'] == 1].copy()

    #calculamos residual absoluto promedio por cliente y cluster
    resumen = (
        df_outliers.groupby(['Numero_Cliente', 'cluster_dbscan'])['Residual']
        .apply(lambda x: np.mean(np.abs(x)))
        .reset_index(name='Residual_Promedio_Abs')
    )

    #Definimos el riesgo de que un filamento de outlier sea una fuga de gas con respecto a su residual promedio por cuantiles
    def asignar_riesgo(grupo):
        p33 = grupo['Residual_Promedio_Abs'].quantile(0.33)
        p66 = grupo['Residual_Promedio_Abs'].quantile(0.66)

        grupo = grupo.copy() # evita errores de asignación en pandas nuevos
        grupo['Riesgo'] = grupo['Residual_Promedio_Abs'].apply(

```

```
        lambda valor: 'Bajo' if valor <= p33 else 'Medio' if valor <= p66 else 'Alto'
    )
    return grupo

resultado = (
    resumen.groupby('Numero_Cliente', group_keys=False)
    .apply(asignar_riesgo)
    .reset_index(drop=True)
)

#unimos para los filamentos/clusters de outliers y agregamos el residual promedio
df = df.merge(
    resultado[['Numero_Cliente', 'cluster_dbscan', 'Riesgo', 'Residual_Promedio_Abs']],
    on=['Numero_Cliente', 'cluster_dbscan'],
    how='left'
)

# Opcional: Rellenar NaN con "Sin riesgo"
df['Riesgo'] = df['Riesgo'].fillna("Sin riesgo")

return df

df_resultado_modelo_4_riesgo=riesgo_cluster(df_resultado_modelo_4)

df_resultado_modelo_4_riesgo

df_resultado_modelo_4_riesgo[df_resultado_modelo_4_riesgo['Numero_Cliente'] == 'CLIENTE11']

df_resultado_modelo_4_riesgo.to_excel("resultado_modelo_4_riesgo.xlsx", index=True)
```

Start coding or [generate](#) with AI.